

Figure 1: Hardware architecture

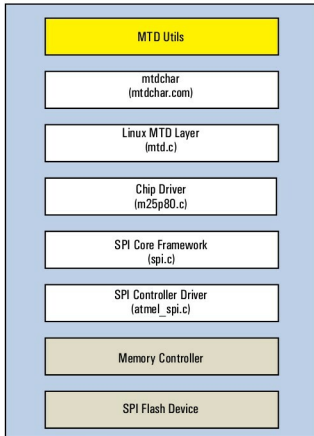


Figure 2: Software architecture

SPI controller driver parts

The SPI controller driver described here consists of three parts:

- A board mapping driver
- A PCI driver

- An SPI slave driver

The board mapping driver has the responsibility of registering flash chip information with the SPI framework. The PCI driver has the responsibility of identifying the SPI controller on the PCH, initialising things like memory mapping or interrupt registration. The SPI slave driver is responsible for initialising and communicating with the flash chip.

SPI controller driver: Initialisation

The board mapping driver and PCI driver mentioned earlier take care of initialisation. The initialisation includes the following activities, at the very least:

- Flash chip information registration to the SPI framework
- SPI master registration to the SPI framework
- SPI slave device creation by the SPI framework
- Registration to the MTD sub-system

Flash chip information registration to the SPI framework

The board mapping driver is responsible for registering flash information with the SPI framework. Ideally, this initialisation should be done in the `arch/x86/` directory during early initialisation. This information is persistent until system reboot. Therefore, if this information is registered by a kernel module, then the module must not be removed and re-inserted. The easiest way to do this is to write a module without the `cleanup_module()` function. Since this information is part of system initialisation, the board mapping driver module must be loaded before the SPI controller and chip driver.

Role of board mapping driver is limited to register partition information. As shown in the code snippet, the board mapping driver registers partition information using `spi_register_board_info()` function.

```

struct mtd_partition pch_spi_flash_partitions[] = {
    {
        .name = "partition1",
        .offset = 0, /* Offset = 0x0 */
        .size = 4 * 1024 * 1024,
    },
    {
        .name = "partition2",
        .offset = MTDPART_OFST_APPEND, /* Offset = 4MB */
        .size = MTDPART_SIZ_FULL,
    },
};

const struct flash_platform_data pch_spi_flash = {
    .name = "spi_flash",
    .parts = pch_spi_flash_partitions,
    .nr_parts = ARRAY_SIZE(pch_spi_flash_partitions),
    .type = "w25q64",
};
    
```